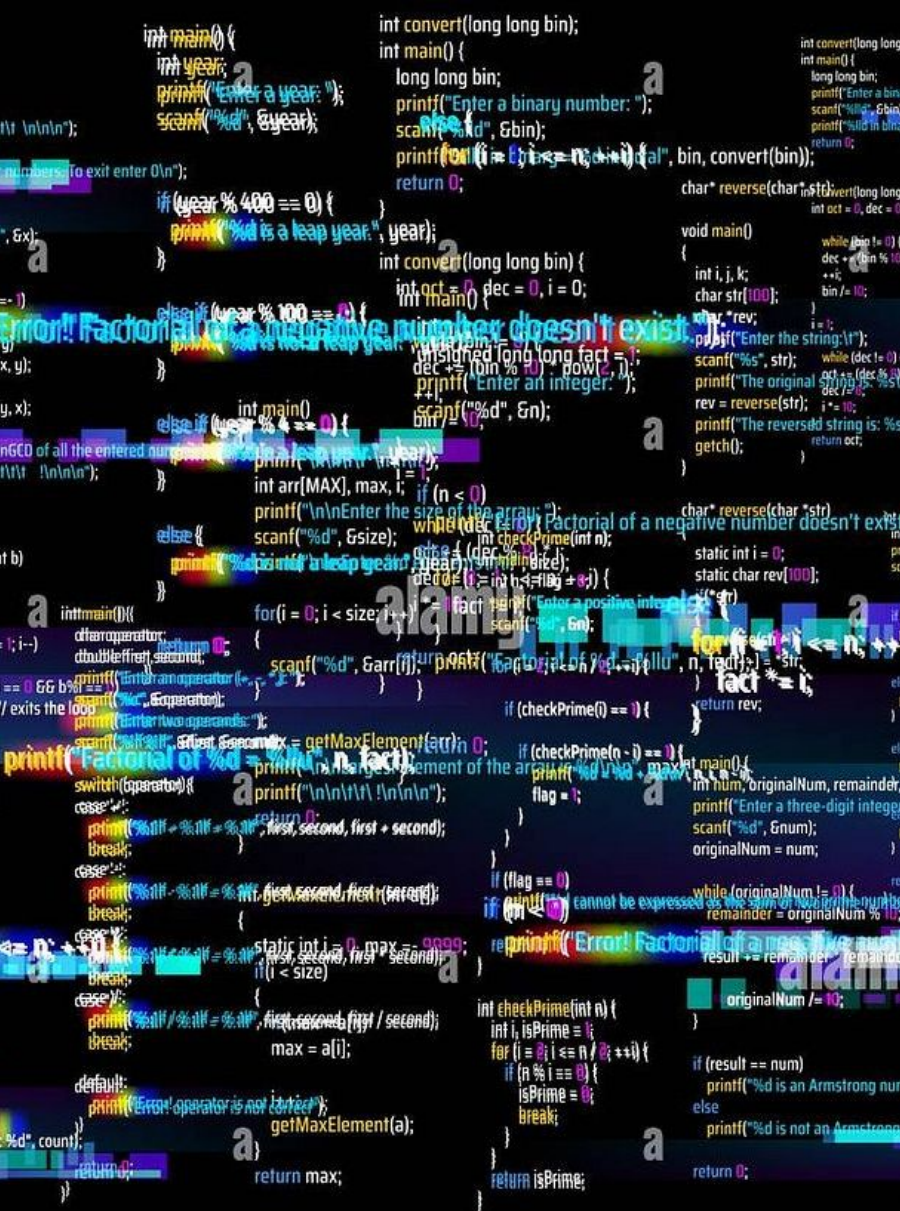




Static Functions & Operator Overloading

Week 13 · Lectures 25 & 26 — C++ Programming

W13 · L25-L26

What Are Static Functions?

Core Definition

A **static member function** is a function that belongs to the **class itself** rather than to any specific object (instance) of that class. It can be called without creating an object of the class, making it a utility or helper function tied to the class namespace. Static functions are declared using the **static** keyword inside the class definition.

Key Characteristics

- Declared with the **static** keyword in the class
- Can be called using the class name and scope resolution operator (**::**)
- Cannot access non-static (instance) members directly
- Do not have a **this** pointer
- Can only directly access other static members (variables or functions)
- Useful for utility functions, factory methods, and shared operations

Static Functions: Syntax & Declaration

Static functions are declared inside the class with the `static` keyword. They are typically defined outside the class (in the `.cpp` file) without repeating the `static` keyword in the definition.

Declaration (Inside Class)

```
class MyClass {  
public:  
    static void displayCount();  
    static int getCount();  
private:  
    static int count;  
};
```

Definition (Outside Class)

```
int MyClass::count = 0;  
  
void MyClass::displayCount() {  
    cout << "Count: " << count <<  
    endl;  
}  
  
int MyClass::getCount() {  
    return count;  
}
```

Calling a Static Function

```
int main() {  
    // No object needed  
    MyClass::displayCount();  
    int c = MyClass::getCount();  
    return 0;  
}
```

Accessing Static Functions

Two Ways to Call Static Functions

Static functions can be accessed in two ways. The recommended and most common approach is using the **class name with the scope resolution operator**. However, they can also be called through an object, though this is less common and can be misleading.

Via Class Name

```
MyClass::staticFunc();
```

Preferred method — clearly shows the function belongs to the class.

Via Object

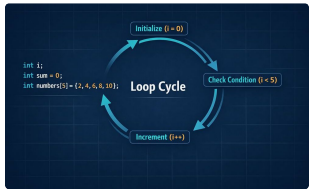
```
MyClass obj;  
obj.staticFunc();
```

Allowed but discouraged — can be confusing since no object state is used.

Why Static Functions Cannot Access Non-Static Members

Non-static (instance) members belong to a specific object and require a `this` pointer to access. Since static functions are not associated with any particular object, they have **no**

Static Functions: Practical Use Cases



Counting Objects

A classic use of static functions is tracking the number of objects created. A static counter variable is incremented in the constructor and decremented in the destructor, while a static function returns the current count.



Factory Methods

Static functions can act as **factory methods** — they create and return objects of the class, often with specific configurations. This is common in design patterns like Singleton or Factory.



Utility / Helper Functions

Functions that perform operations related to the class but don't need object state — such as mathematical conversions, validation, or formatting — are ideal candidates for static functions.

The operator Keyword

What Is Operator Overloading?

Operator overloading allows you to **redefine or overload** most built-in operators so they work with user-defined types (classes/structs). The `operator` keyword is used to declare an operator function. This enables objects of your class to be used with familiar operators like `+`, `-`, `==`, `<<`, etc., making code more intuitive and readable.

General Syntax

```
returnType operator symbol (parameters) {  
    // function body  
}
```

For example, to overload the `+` operator:

```
MyClass operator+(const MyClass& other) {  
    MyClass temp;  
    temp.value = this->value + other.value;  
    return temp;  
}
```

Key Rules

- Cannot change the number of operands an operator takes
- Cannot create entirely new operators
- Cannot change the precedence or associativity of operators
- Some operators cannot be overloaded (e.g., `::`, `.*`, `?:`)

Types of Operator Overloading

Operators can be overloaded as either **member functions** or **non-member (friend) functions**. The choice depends on whether the left-hand operand is an object of the class.



As a Member Function

The left operand is implicitly the calling object (`this`). Used when the left-hand side is an object of the class. Example: `obj1 + obj2` — here `obj1` calls the operator function.

```
MyClass operator+(const MyClass&
rhs) {
    // this->value + rhs.value
}
```



As a Friend Function

Used when the left operand is **not** an object of the class (e.g., overloading `<<` for `ostream`). A friend function has access to private members but is not a member of the class.

```
friend ostream&
operator<<(ostream& os, const
MyClass& obj);
```



As a Non-Member Function

Similar to friend but without needing access to private members. Used when the operator is symmetric and neither operand needs special access. Often used for commutative operations.

```
MyClass operator+(const MyClass&
lhs, const MyClass& rhs);
```

Overloading Unary Operators

Unary operators operate on a **single operand**. Common unary operators include `++` (increment), `--` (decrement), `-` (negation), `!` (logical NOT), and `*` (dereference). When overloading unary operators as member functions, they take **no parameters** (for prefix) or a dummy `int` parameter (for postfix).

Prefix vs. Postfix Increment

The **prefix** form (`++obj`) increments first, then returns the updated value. The **postfix** form (`obj++`) returns the original value, then increments. C++ distinguishes them by adding a dummy `int` parameter for postfix.

```
// Prefix ++obj
MyClass& operator++() {
    value++;
    return *this;
}

// Postfix obj++
MyClass operator++(int) {
    MyClass temp = *this;
    value++;
    return temp;
}
```

Other Unary Operators

```
// Unary minus (negation)
MyClass operator-() {
    MyClass temp;
    temp.value = -value;
    return temp;
}

// Logical NOT
bool operator!() {
    return (value == 0);
}

// Address-of operator
MyClass* operator&() {
    return this;
}
```

- ❏ Return `*this` by reference for prefix operators to allow chaining. Return by value for postfix to preserve the original state before increment.

Complete Example: Static Functions & Operator Overloading

The following example demonstrates both concepts together — a `Counter` class with a static function to track object count, and overloaded unary operators for increment/decrement.

```
class Counter {
private:
    int value;
    static int objectCount; // static member variable

public:
    Counter(int v = 0) : value(v) {
        objectCount++;
    }

    // Static function - returns total objects created
    static int getObjectCount() {
        return objectCount;
    }

    // Prefix increment: ++obj
    Counter& operator++() {
        value++;
        return *this;
    }

    // Postfix increment: obj++
    Counter operator++(int) {
        Counter temp = *this;
        value++;
        return temp;
    }

    // Unary minus: -obj
    Counter operator-() {
        Counter temp;
        temp.value = -value;
        return temp;
    }

    void display() {
        cout << "Value: " << value << endl;
    }
};

int Counter::objectCount = 0; // static member initialization

int main() {
    Counter c1(5);
    Counter c2(10);

    ++c1; // prefix
    c2++; // postfix
    c1.display(); // Value: 6
    c2.display(); // Value: 11

    Counter c3 = -c1;
    c3.display(); // Value: -6

    cout << "Total objects: " << Counter::getObjectCount() << endl; // 3
    return 0;
}
```

Key Takeaways



Static Functions Belong to the Class

Static functions are associated with the class, not any object. Call them using `ClassName::function()`. They cannot access non-static members directly and have no `this` pointer.



Two Ways to Access Static Functions

Preferred: via class name (`MyClass::func()`). Also allowed (but discouraged): via object (`obj.func()`). Static functions are ideal for utility methods, factory methods, and tracking shared state.



The operator Keyword

Use `operator` to overload built-in operators for your classes. Operators can be overloaded as member functions (left operand = `this`) or friend/non-member functions (for external left operands).



Unary Operator Overloading

Prefix unary operators take no parameters and return `*this` by reference. Postfix unary operators take a dummy `int` parameter and return by value. Common unary operators: `++`, `--`, `-`, `!`.

 **Remember:** Static functions + operator overloading are powerful C++ features that, when combined, allow you to write clean, expressive, and efficient code that behaves naturally with user-defined types.